

# VibecoSwagaTemplate

Шаблон для соло-вайбкодера

Все передовые практики Claude Code в одном `Use this template`.

2026 · MIT

# Зачем этот шаблон существует

LLM-агенты системно ошибаются одинаково. Их **надо учить хорошим привычкам через** `CLAUDE.md` — иначе они:

- Делают допущения за тебя и идут с ними молча
- Не управляют своей confusion, не задают вопросов
- Переусложняют код и раздувают абстракции
- Не чистят за собой мёртвый код
- Правят рядом с задачей то, что трогать не просили

**Шаблон закладывает противоядие сразу.** Клонировешь → описываешь идею → дальше Claude уже знает как себя вести.

# Диагноз Karpathy (Jan 2026)

“ «Модели делают неправильные допущения на твой счёт и просто идут с ними. Они не управляют своей *confusion*, не просят уточнений, не подсвечивают противоречия, не показывают *tradeoffs*, не пушат обратно когда должны. Они переусложняют код, раздувают абстракции, не чистят мёртвый код».

”

— Andrej Karpathy, [твит про Claude Code](#)

# Решение: 4 принципа в CLAUDE.md

| Принцип                   | Лечит                                             |
|---------------------------|---------------------------------------------------|
| 1. Думай до кода          | Допущения, скрытая confusion, упущенные tradeoffs |
| 2. Минимум кода           | Переусложнение, раздутые абстракции               |
| 3. Хирургические правки   | Орто-правки, трогает то, что не должен            |
| 4. Goal-driven исполнение | Слабые критерии → нужны постоянные уточнения      |

Источник интерпретации: [forrestchang/andrey-karpathy-skills](https://forrestchang.github.io/andrey-karpathy-skills/). Адаптировано на русский + добавлены локальные практики.

# 1. Думай до кода

**Не предполагай. Не прячь свою confusion. Поднимай tradeoffs.**

- Проговори допущения вслух. Не уверен — **спроси**.
- Несколько интерпретаций — **назови все**, не выбирай молча.
- Видишь более простой путь — **скажи**. Push back.
- Никаких "of course!" в ответ на критику. Прав → фикси. Не прав → аргументируй.

## 2. Минимум кода

**Только то, что решает задачу. Ничего на будущее.**

- Никаких фич сверх запрошенного.
- Никаких абстракций для одноразового кода.
- Никакой обработки невозможных ошибок.
- 200 строк, а можно 50? → **перепиши**.
- **Regex до LLM**: парсинг структурированного текста — regex, LLM только когда структура нерегулярная.

# 3. Хирургические правки

**Трогай только то, что должен. Чисти только за собой.**

- Не «улучшай» соседний код, комментарии, форматирование.
- Не рефактори то, что не сломано.
- Заметил несвязанный мёртвый код — **скажи**, не удаляй.
- Убирай только те хвосты, которые создали твои изменения.

**Тест:** каждая изменённая строка должна прямо трассироваться к запросу пользователя.

## 4. Goal-driven исполнение

**Определи критерий успеха. Крути цикл, пока не зелёный.**

- «Добавь валидацию» → «Тесты на невалидные входы, потом сделай их зелёными».
- «Почини баг» → «Тест воспроизводит, потом фикс, тест зелёный».
- «Зарефактори» → «Тесты зелёные до и после».

Сильные критерии = агент крутит цикл сам.

Слабые («сделай чтобы работало») = постоянные уточнения.



# Локальные практики (поверх Karpathy)

| Раздел                        | Суть                                                                                                                               |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| §3 UI первой                  | Любая фича/проект с UI начинается с HTML-мокапа через <code>frontend-design</code> или <code>gsd-sketch</code> . <b>Потом</b> бэк. |
| §4 Docker                     | Тесты, запуски — только в контейнерах. Никакого "у меня локально работает".                                                        |
| §5 Тесты                      | Integration с реальной БД. Без mock. TDD: RED → GREEN → REFACTOR → COMMIT.                                                         |
| §6 Research-first + Stop rule | 5 минут <code>grep</code> / <code>mem-search</code> / <code>context7</code> перед сложным. Три провала → стоп.                     |
| §7 Секреты                    | Через <code>.env</code> + <code>.env.example</code> . Никакого <code>--no-verify</code> .                                          |
| §8 Контекст                   | Большой вывод → <code>context-mode</code> . <code>/compact</code> по breakpoints, не по 95%.                                       |

## §3: UI — сначала набросок, потом код

**90% переделок бэка случаются потому, что UI оказался не таким.**

Старт фичи с экраном:

1. frontend-design / gsd-sketch  
→ HTML-мокап (throwaway, 30 мин)
2. Обсуждение с пользователем  
→ "да, такое" / "нет, переделай"
3. Контракт API под этот UI
4. Имплементация (бэк + фронт)

Дешевле перерисовать мокап, чем мигрировать схему.

# Стек плагинов Claude Code

| Плагин                    | Когда работает                                           |
|---------------------------|----------------------------------------------------------|
| <b>superpowers</b>        | Дисциплина задачи: brainstorm, plans, TDD, debugging     |
| <b>gsd</b> ( <i>opt</i> ) | Многофазные проекты, артефакты в <code>.planning/</code> |
| <b>claude-mem</b>         | Память между сессиями: <code>mem-search</code>           |
| <b>context7</b>           | Свежие доки библиотек, не галлюцинации                   |
| <b>context-mode</b>       | Защита контекстного окна от больших выводов              |
| <b>frontend-design</b>    | Дизайн-система + HTML-мокапы для §3                      |
| <b>reflexion</b>          | Мульти-перспективная критика после крупной фичи          |
| <b>github</b>             | <code>gh</code> операции из чата                         |

# Quick start

```
# 1. Use this template на GitHub → clone  
git clone git@github.com:<you>/<project>.git  
cd <project>
```

```
# 2. Один раз: включи плагины глобально  
./scripts/install-plugins.sh
```

```
# 3. Запусти Claude и опиши идею  
claude  
> хочу маркетплейс кроссовок с auth и Stripe
```

# Что делает `install-plugins.sh`

Нативный путь — никаких фейков:

1. **Бэкап** `~/.claude/settings.json` (timestamp в имени).
2. `jq -merge enabledPlugins map: {name@marketplace: true}`.
3. Выводит список marketplaces для **одноразовой** ручной регистрации внутри `claude` ( `/plugin marketplace add thedotmack/claude-mem` и т.п.).

Claude Code на старте сам подтянет плагины и держит их обновлёнными.

Никаких `npm install` per-project, никаких per-репо плагинов — всё глобально.

# Структура шаблона

```
.
├── CLAUDE.md           правила Claude Code
                        (Karpathy + локальные практики)
├── README.md           старт
├── .claude-plugins.json список плагинов с описанием
├── scripts/
│   └── install-plugins.sh jq-мерджит enabledPlugins
                        в ~/.claude/settings.json
├── docs/
│   ├── presentation.md эта презентация
│   ├── system-design-patterns.md шпаргалка Alex Xu
│   └── gsd-vs-superpowers.md GSD ↔ superpowers
```

Никакого backend/frontend кода. Стек выбираешь сам или с Claude.

# Источники

- [karpathy/status/2015883857489522876](https://karpathy/status/2015883857489522876) — диагноз болезней LLM-кода
- [forrestchang/andrey-karpathy-skills](https://forrestchang/andrey-karpathy-skills) — 4 принципа в одном CLAUDE.md
- [affaan-m/everything-claude-code](https://affaan-m/everything-claude-code) — донор для research-first, regex-vs-LLM, strategic /compact
- **superpowers + gsd + claude-mem** — workflow plugins
- **Alex Xu, System Design Interview Vol. 1** — шпаргалка в docs/

# Поехали

```
git clone git@github.com:<you>/<project>.git  
cd <project>  
./scripts/install-plugins.sh  
claude
```

[github.com/timderbak/VibecoSwagaPlatform](https://github.com/timderbak/VibecoSwagaPlatform) · MIT